

TITLE

**CLOSED-LOOP REAL-TIME PHYSIOLOGICAL PERFORMANCE OPERATING
SYSTEM AND METHODS OF USE**

BACKGROUND

- 5 [0001] The invention generally relates to digital health systems, physiological control systems, performance optimization platforms, and real-time behavioral command systems.
- [0002] Prior art systems including WHOOP only have passive tracking, Apple Health only has data aggregation, Garmin only has performance metrics, and hydration products provide static consumption. None of the prior art systems provide: Autonomous command execution, Product-
10 linked intake integration, Deterministic AI output, and a Closed-loop behavioral system. The prior art systems only include passive tracking, no real-time action generation, no integration with consumable products, no closed-loop feedback, and no compliance-driven execution. Existing systems still fail to: convert data into behavior, provide real-time intervention, and create continuous adaptive loops.
- 15 [0003] The present invention attempts to solve these problems, as well as others.

SUMMARY OF THE INVENTION

- [0004] Provided herein are systems, methods and apparatuses for a closed-loop real-time physiological performance operating system.
- [0005] The methods, systems, and apparatuses are set forth in part in the description which
20 follows, and in part will be obvious from the description, or can be learned by practice of the methods, apparatuses, and systems. The advantages of the methods, apparatuses, and systems will be realized and attained by means of the elements and combinations particularly pointed out in the appended claims. It is to be understood that both the foregoing general description and the following detailed description are exemplary and explanatory only and are not restrictive of the
25 methods, apparatuses, and systems, as claimed.
- [0006] Accordingly, it is an object of the invention not to encompass within the invention any previously known product, process of making the product, or method of using the product such that Applicants reserve the right and hereby disclose a disclaimer of any previously known product, process, or method. It is further noted that the invention does not intend to encompass within the
30 scope of the invention any product, process, or making of the product or method of using the product, which does not meet the written description and enablement requirements of the USPTO

(35 U.S.C. § 112, first paragraph) or the EPO (Article 83 of the EPC), such that Applicants reserve the right and hereby disclose a disclaimer of any previously described product, process of making the product, or method of using the product. It may be advantageous in the practice of the invention to be in compliance with Art. 53(c) EPC and Rule 28(b) and (c) EPC. All rights to explicitly
5 disclaim any embodiments that are the subject of any granted patent(s) of applicant in the lineage of this application or in any other lineage or in any prior filed application of any third party is explicitly reserved. Nothing herein is to be construed as a promise.

BRIEF DESCRIPTION OF THE DRAWINGS

[0007] In the accompanying figures, like elements are identified by like reference numerals among
10 the several preferred embodiments of the present invention.

[0008] **FIG. 1** is a schematic flow diagram of the closed-loop real-time physiological performance operating system, according to one embodiment.

[0009] **FIG. 2** is a schematic flow diagram of the closed-loop real-time physiological performance operating system architecture, according to one embodiment.

15 [0010] **FIG. 3** is a schematic flow diagram of the closed-loop real-time physiological performance operating system architecture, according to one embodiment.

[0011] **FIG. 4** is a system implementation of the closed-loop real-time physiological performance operating system, according to one embodiment.

[0012] **FIG. 5** is a schematic flow diagram of the System Architecture including OS mobile client,
20 the api-server, the Postgres data store, and the WebSocket real-time channel that synchronizes state across devices.

[0013] **FIG. 6** is a schematic flow diagram of the Mobile App Architecture, including the Sliced React Context + pure reducer + memoized slice projections so unrelated UI sections do not re-render on unrelated state changes, using expo-router file-based navigation. Clerk auth bridge
25 mediates between Clerk session and the internal store and Apple HealthKit signals enter the engine through the appleHealth field of UserState.

[0014] **FIG. 7** is a diagram of the wireframes from the Software Application including five primary tabs of AForce OS — Hydration Control Center (home), Performance Signals (check), Recovery Protocol, Store, and Profile — plus the Sweat Calculator and Autoscan modal entry points.

30 [0015] **FIG. 8** — Scoring Engine. Pure-function pipeline from UserState snapshot through (a) per-event absorption, (b) bounded contribution sum, (c) continuous decay model, to (d) score / band /

breakdown / prediction /command. Server- and client-side execution produce identical output for identical input ("score identity invariant").

[0016] FIG. 9 is a schematic flow diagram of the Deterministic AI Layer, which is an ordered rule pipeline: mode classification → penalty application → bonus application → decay projection → command generation → recheck cadence. No remote LLM inference in the safety- and performance-critical path; the same snapshot always produces the same command. Safetyclass commands (Social Mode, Guardian Risk Engine) preempt the standard band ladder.

[0017] FIG. 10 is a schematic flow diagram of the Social Mode Flow including Drink-logging panel → Widmark BAC estimate (parameterized by sex, weight, drinks, ABV, time elapsed, food intake) → impairment-level mapping (LOW / ELEVATED / MODERATE / HIGH / CRITICAL) → safety-class command override that preempts the standard performance ladder.

[0018] FIG. 11 is a schematic flow diagram of the Autopilot / Autoscan, where the Autopilot is the top half: sweat session → deficit % → autopilot cadence ladder ($\geq 4\%$ → 8 min / critical, $\geq 2\%$ → 12 min / high, else → 20 min / moderate, 4 h window) → atomic state transition that simultaneously stores the autopilot tuple, resets the visible recheck timer, and clears any pending confirmation prompt. Autoscan is Bottom half: barcode / QR scan → product recognition (local catalog → Open Food Facts fallback) → live-state fit score → conditional AForce replacement decision when brand fit exceeds scanned fit by > 4 points.

DETAILED DESCRIPTION OF THE INVENTION

[0019] The foregoing and other features and advantages of the invention are apparent from the following detailed description of exemplary embodiments, read in conjunction with the accompanying drawings. The detailed description and drawings are merely illustrative of the invention rather than limiting, the scope of the invention being defined by the appended claims and equivalents thereof.

[0020] Embodiments of the invention will now be described with reference to the Figures, wherein like numerals reflect like elements throughout. The terminology used in the description presented herein is not intended to be interpreted in any limited or restrictive way, simply because it is being utilized in conjunction with detailed description of certain specific embodiments of the invention. Furthermore, embodiments of the invention may include several novel features, no single one of

which is solely responsible for its desirable attributes or which is essential to practicing the invention described herein.

[0021] The use of the terms “a” and “an” and “the” and similar referents in the context of describing the invention are to be construed to cover both the singular and the plural, unless
5 otherwise indicated herein or clearly contradicted by context. It will be further understood that the terms “comprises,” “comprising,” “includes,” and/or “including,” when used herein, specify the presence of stated features, integers, steps, operations, elements, and/or components, but do not preclude the presence or addition of one or more other features, integers, steps, operations, elements, components, and/or groups thereof.

[0022] Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within the range, unless otherwise indicated
10 herein, and each separate value is incorporated into the specification as if it were individually recited herein. The word “about,” when accompanying a numerical value, is to be construed as indicating a deviation of up to and inclusive of 10% from the stated numerical value. The use of
15 any and all examples, or exemplary language (“e.g.” or “such as”) provided herein, is intended merely to better illuminate the invention and does not pose a limitation on the scope of the invention unless otherwise claimed. No language in the specification should be construed as indicating any nonclaimed element as essential to the practice of the invention.

[0023] References to “one embodiment,” “an embodiment,” “example embodiment,” “various
20 embodiments,” etc., may indicate that the embodiment(s) of the invention so described may include a particular feature, structure, or characteristic, but not every embodiment necessarily includes the particular feature, structure, or characteristic. Further, repeated use of the phrase “in one embodiment,” or “in an exemplary embodiment,” do not necessarily refer to the same embodiment, although they may.

[0024] As used herein the term “method” refers to manners, means, techniques and procedures for
25 accomplishing a given task including, but not limited to, those manners, means, techniques and procedures either known to, or readily developed from known manners, means, techniques and procedures by practitioners of the chemical, biological, software, hardware, and electrical arts. Unless otherwise expressly stated, it is in no way intended that any method or aspect set forth
30 herein be construed as requiring that its steps be performed in a specific order. Accordingly, where a method claim does not specifically state in the claims or descriptions that the steps are to be

limited to a specific order, it is no way intended that an order be inferred, in any respect. This holds for any possible non-express basis for interpretation, including matters of logic with respect to arrangement of steps or operational flow, plain meaning derived from grammatical organization or punctuation, or the number or type of aspects described in the specification.

5 **[0025] Description of Embodiments**

[0026] Generally speaking, a closed-loop real-time physiological performance operating system comprises: continuously monitoring a physiological state, computing a dynamic performance score; delivering compliance-driven commands; and operating without requiring active user initiation. The system integrates: real-time physiological and behavioral inputs; a multi-variable
10 scoring engine; a deterministic AI command enforcement layer; an autonomous execution engine (“AutoPilot”); and a product-linked intake system (“HydroScan”). The system is a proactive physiological control system that continuously evaluates human physiological state and delivers structured behavioral commands in real time.

[0027] In one embodiment, the comprises a software system that produces real-time hydration risk
15 assessments and prescriptive recommendations for cruise-ship guests, using a fusion of (a) a static guest-persona profile, (b) live per-port environmental data resolved against a registry of cruise terminal coordinates, and (c) a maritime-specific scoring model that incorporates variables (sun exposure, alcohol intake, deck exposure, port-day excursion) absent from terrestrial hydration tools. The system further employs a three-tier resilience strategy ensuring deterministic, plausible
20 output under network or upstream failure.

[0028] As shown in **Fig. 1**, the closed-loop real-time physiological performance operating system
100 components comprise an input layer 110, a scoring engine 120, a deterministic AI layer 130, an AutoPilot system 140, and a HydroScan integration 150. The Input Layer 110 comprises at least one biometric data 112 (heart rate, activity, sweat indicators, and the like), behavioral data 114
25 (timing, habits, compliance, and the like), at least one environmental data (temperature, humidity, and the like), and at least one intake event 116 (manual or product-linked). The Scoring Engine 120 comprises computing a dynamic performance score 122 (0–100) and calculating a confidence level and rate of change 124. The Deterministic AI Layer 130 comprises validating all outputs 132 and enforcing a structured command logic 134. The AutoPilot System 140 comprises delivering
30 commands automatically 142 and adjusting the timing and urgency dynamically. The HydroScan Integration 150 comprises linking physical product usage to digital intake logging.

[0029] In one embodiment, the closed-loop real-time physiological performance operating system optimizes athletic performance optimization, enhances cognitive performance, manages fatigue and recovery, optimizes daily physiological, or provides behavioral compliance systems.

[0030] As shown in **Fig. 2**, the closed-loop real-time physiological performance operating system architecture 200 comprises a Capture Layer 210, an Analysis and Scoring Engine 220, a Deterministic AI Command Layer 230, an AutoPilot Execution System 240, and a Product-Linked Intake System 250 (HydroScan). The Capture Layer 210 collects Physiological inputs 212, Behavioral patterns 214, Environmental conditions 216, and Intake events 218 (manual or scanned). The Analysis & Scoring Engine 220 processes inputs 222 using weighted models to determine: Hydration state, Physiological stress, Behavioral compliance, Performance readiness.

[0031] The Analysis & Scoring Engine 220 processes outputs 224 including: Performance Score (0–100), Confidence score, Rate-of-change indicators. The Deterministic AI Command Layer 230 generates structured outputs 232 including: Action (e.g., drink, hydrate, recover), Timing (immediate, scheduled, delayed), Expected outcome. The Deterministic AI Command Layer 230 ensures: Non-conversational output, Standardized format, Compliance-driven instructions. The AutoPilot Execution System 240 comprises continuously monitoring physiological state 242, triggering commands without user initiation 244, adjusting command frequency and urgency 246. The AutoPilot Execution System 240 converts the system into a proactive physiological control engine. The Product-Linked Intake System (HydroScan) 250 comprises: QR / NFC scanning of consumable products 252, an automatic intake event generation 254, conversion into physiological units 256, and an Immediate recalculation of performance score 258.

[0032] After action execution of the closed-loop real-time physiological performance operating system architecture 200, the Closed-Loop Feedback System 260 monitors physiological response 262, updates performance score 264, and adjusts future commands 266. The Closed-Loop Feedback System 260 creates a feedback loop from input, score, command, action, feedback, and recalculation.

[0033] As shown in **Fig. 3**, the closed-loop real-time physiological performance operating system architecture 200 further comprises a plurality of Expanded Inputs 270 (Optional Embodiments), a Behavioral Reinforcement Layer 280, and a Network & Community Layer 290. The plurality of Expanded Inputs 270 comprises a urine color detection via camera 272, an image-based hydration classification 274, or a machine learning image recognition. The Behavioral Reinforcement Layer

280 includes: Compliance scoring, Challenges (individual + group), Rewards and feedback, or
Streak tracking 282. The Network & Community Layer 290 includes: Shared challenges,
Performance comparison, Group optimization, or data aggregation for system improvement 292.

[0034] The closed-loop real-time physiological performance operating system converts data into
5 action for the user, integrates product and the user's behavior, operates in real time, and creates a
continuous feedback loop. In one embodiment, the system is a closed-loop hydration-performance
system, provides real-time performance scoring, provides action-based output (not just tracking),
and integrates with consumable products. In one embodiment, the system comprises a behavioral
optimization layer, a visual hydration detection via urine color analysis, behavioral reinforcement
10 through gamification mechanisms, or a community-based hydration challenge and engagement
system.

[0035] Alternative Embodiments

[0036] 1. The method and system comprises a pure-function pipeline computing a per-minute
decaying performance score from a defined set of biometric, environmental, and behavioral inputs,
15 with a server- and client-side identity invariant (same input $\Rightarrow$ same output).

[0037] 2. The method and system comprises a sweat-deficit-driven cadence override that replaces
a band-based recheck timer with a physiologically-derived interval ladder (8/12/20 min) for a
bounded recovery window (4 h) following a measured sweat session.

[0038] 3. The method and system comprises a beverage-recognition pipeline that scores any
20 scanned product against a live user state (not a static nutrition database) and conditionally surfaces
a brand-specific replacement when the brand's fit score exceeds the scanned product's fit score by
a configurable margin.

[0039] 4. The method and system comprises an audit-on-output: every score returned to the UI
carries a breakdown[] array of bounded contributions so the user can inspect why the score is what
25 it is.

[0040] 5. The method and system comprises a mode-priority pipeline in which a Social Mode
ruleset (BAC + impairment) preempts the standard performance ladder and emits safety-class
commands that the rest of the engine cannot override.

[0041] Pure-function performance scoring with audit-on-output

30 [0042] In one embodiment, the Pure-function performance scoring with audit-on-output system
comprises a scoring system that computes a numeric performance score for a hydration / recovery

user from a defined snapshot of inputs, where the same input snapshot always produces the same output, and where every score returned to the UI is accompanied by a structured breakdown of bounded contributions.

[0043] The Pure-function performance scoring with audit-on-output system comprises a user-state snapshot data structure comprising at least: cumulative fluid intake on a per-event basis with timestamps; cumulative AForce-formulated unit count; environmental load (heat, humidity, activity); a physiological self-report axis; one or more biometric samples (HRV, sleep) read from a device-level health framework; and a boolean operating-mode flag (e.g. clutch / social).

[0044] The Pure-function performance scoring with audit-on-output system comprises a pure-function score engine that consumes the snapshot and produces a numeric score in a fixed range (e.g. 0 – 100) by summing a defined set of bounded contribution terms.

[0045] The Pure-function performance scoring with audit-on-output system comprises a continuous-time decay model applied to the score between intake events, parameterized by user mass, activity, and environment.

[0046] The Pure-function performance scoring with audit-on-output system comprises a breakdown array, returned alongside the score on every invocation, in which each entry comprises an identifier, a human-readable label, a signed delta, and a maximum-magnitude bound, such that no single contribution can exceed its bound.

[0047] The Pure-function performance scoring with audit-on-output system comprises Server-side and client-side execution of the same engine on the same snapshot, producing the same score and the same breakdown (a "score identity invariant").

[0048] The Pure-function performance scoring with audit-on-output system comprises a score function, a decay function, a breakdown shape, a per-event absorption, and a server-replay invariant. The server-replay invariant re-runs the same Scoring Engine on every API response. The audit-on-output pattern means every score the user sees can be reproduced offline by replaying the snapshot, which is a property required for safety-class behavior and for a meaningful UI.

[0049] Pure-function performance scoring with audit-on-output system further comprises the breakdown is rendered to the user as a per-term magnitude bar chart. Pure-function performance scoring with audit-on-output system further comprises each contribution term is derived from a single named input axis, such that the user can trace any score change to a specific change in their behavior. Pure-function performance scoring with audit-on-output system further comprises the

decay model is multiplied by an operating-mode multiplier when the boolean operating-mode flag is set.

[0050] Sweat-deficit-driven cadence override (Autopilot)

[0051] The Sweat-deficit-driven cadence override (Autopilot) method is for replacing a band-based recheck timer with a physiologically-derived recheck interval for a bounded recovery window after a measured sweat session. Hydration coaching apps prompt the user to log fluid intake at fixed or rule-of thumb intervals (e.g. every 30 minutes). These intervals are not aware of the user's actual fluid loss during a measured activity, and therefore systematically under-prompt high-deficit athletes during the post activity period when corrective intake matters most.

[0052] The Sweat-deficit-driven cadence override (Autopilot) method comprises a sweat-rate engine that consumes a session description (mode, intake, output, environment, duration) and produces at least: a sweat rate, a hydration deficit expressed as a percentage of body mass, and a sodium-loss estimate.

[0053] The Sweat-deficit-driven cadence override (Autopilot) method comprises a cadence-derivation function that maps the deficit percentage to an (interval, urgency, window) tuple via a stepped table: a first deficit threshold mapping to a short interval and a critical urgency; a second, lower threshold mapping to a longer interval and a high urgency; otherwise the longest interval and a moderate urgency. The window is fixed.

[0054] The Sweat-deficit-driven cadence override (Autopilot) method comprises an atomic state-transition action ("set autopilot") that, on dispatch, simultaneously: stores the autopilot tuple and a timestamp; resets a visible recheck-countdown to $\text{interval} \times 60$ seconds; and clears any pending user-confirmation prompt.

[0055] The Sweat-deficit-driven cadence override (Autopilot) method comprises a consumer hook that, on each evaluation tick, checks whether the autopilot timestamp lies within the recovery window; if so, returns the autopilot interval and urgency in place of the band-default cadence; if not, returns the band-default cadence.

[0056] The Sweat-deficit-driven cadence override (Autopilot) method comprises a display of the resulting countdown in the same UI element regardless of source, such that the user experience does not change when the cadence source switches.

[0057] The Sweat-deficit-driven cadence override (Autopilot) method is implemented by the Sweat engine + cadence derivation, an Atomic state transition, a Consumer hook, and an UI surface.

[0058] No consumer hydration app the team is aware of overrides its recheck cadence with a per session-derived interval ladder. Existing products use either fixed timers or simple rule-of-thumb heuristics that do not consume a measured deficit %. The atomic reset of the visible timer at the moment of dispatch — so the user immediately sees the new cadence — is also non-trivial: it requires the cadence source and the timer source to be co-mutated in a single transition.

[0059] The Sweat-deficit-driven cadence override (Autopilot) method further comprises the cadence source falls back to the band-default cadence after the recovery window expires, without an explicit exit step. The Sweat-deficit-driven cadence override (Autopilot) method further comprises the recovery window is four hours. The Sweat-deficit-driven cadence override (Autopilot) method further comprises the cadence-derivation function uses thresholds of approximately 4 % and 2 % deficit, mapping to intervals of 8, 12, and 20 minutes respectively.

[0060] Live-state beverage-replacement recommendation (Autoscan)

[0061] Live-state beverage-replacement recommendation (Autoscan) method is a barcode-driven beverage-recognition system that scores a scanned product not against a static nutrition database but against a contemporaneously computed user state, and that surfaces a brand specific replacement recommendation when an in-catalog brand item exceeds the scanned product's fit score by a configurable margin.

[0062] Existing barcode-scanning beverage apps return static nutrition facts (calories, sugar, sodium) regardless of the user's current physiological state. They do not score a scanned product against a live computation of what the user actually needs in the moment, and they do not recommend a specific replacement product.

[0063] Live-state beverage-replacement recommendation (Autoscan) method comprises a camera-and-barcode subsystem capable of resolving common UPC / EAN symbologies and a brandspecific QR scheme; the resolution layer is augmented by an external public-database fallback when a barcode is not present in the local catalog.

[0064] Live-state beverage-replacement recommendation (Autoscan) method comprises a normalized product schema with at least: product identity, brand, category, hydration speed,

electrolyte density, sugar content, and an isBrand flag identifying products from the application's own brand catalog.

[0065] Live-state beverage-replacement recommendation (Autoscan) method comprises a comparison engine that takes the resolved product, a snapshot of the user's current state (the same snapshot consumed by the scoring engine of Hook 1), and produces a fit score (0 – 100) and a discrete verdict.

[0066] Live-state beverage-replacement recommendation (Autoscan) method comprises a replacement-decision rule: filter the product catalog to brand items only; compute the best brand item's fit score against the same user state; if $(\text{brandFitScore} - \text{scannedFitScore}) > T$ (a configurable margin), return a replacement card naming the brand item and a one-line user-actionable command.

[0067] Live-state beverage-replacement recommendation (Autoscan) method comprises a result UI that displays both the scanned product and (conditionally) the replacement card, with a single tap to log either choice as an IntakeEvent.

[0068] Live-state beverage-replacement recommendation (Autoscan) method is implemented with Recognition + OFF fallback, Scan orchestrator, comparison engine, and Camera UI.

[0069] A live-state replacement is a different user experience from a static-facts lookup: the recommendation depends on the user's current heat load, last intake, urine signal, etc., not just on the scanned product. It requires the comparison engine and the live state to share a snapshot type, which in turn requires the scoring system of Hook 1.

[0070] Live-state beverage-replacement recommendation (Autoscan) method further comprises the configurable margin T is approximately 4 fit-score points, the brand-specific QR scheme uses the URI form, or the result UI presents the replacement card only when the scanned product is not itself an in-brand item.

[0071] Mode-priority decision pipeline with safety-class preemption

[0072] Mode-priority decision pipeline with safety-class preemption is a decision pipeline that classifies a user into one of a defined set of operating modes, where a "safety-class" mode (e.g. alcohol-impairment) preempts the standard performance pipeline and emits commands the rest of the engine cannot override. Wellness apps that issue user-facing guidance commonly use a single decision tree across all states. A user who is intoxicated still receives the same hydration nudge as

a sober athlete. There is no architectural mechanism for a safety-related command to take precedence over a routine command.

[0073] Mode-priority decision pipeline with safety-class preemption method comprises a classification step that, given the user-state snapshot, identifies whether a safety-class mode is active (in the implementation: `socialMode.active === true`).

[0074] Mode-priority decision pipeline with safety-class preemption method comprises a safety-class command generator that, when the safety-class mode is active, runs a parallel ruleset whose outputs cannot be overridden by the standard ladder. In the implementation: BAC estimation via the Widmark approximation, impairment-level mapping to one of five tiers, and an escalation table that selects from a set of locked, globally-safe command strings.

[0075] Mode-priority decision pipeline with safety-class preemption method comprises a standard-mode command generator that runs only when the safety-class mode is not active, selecting commands from a band-keyed table.

[0076] Mode-priority decision pipeline with safety-class preemption method comprises a pipeline order in which the safety-class generator runs first; if it returns a non-null command, that command is the engine's output, and the standard generator is not consulted.

[0077] Mode-priority decision pipeline with safety-class preemption method comprises a locked-copy registry for safety-class command strings, separated from the localized template store, such that the disclaimer copy cannot be edited without an explicit code-level change.

[0078] Mode-priority decision pipeline with safety-class preemption method is implemented with a Pipeline order, Safety-class generator, BAC + impairment, and Standard generator.

[0079] The architectural commitment that a safety-class command cannot be overridden by a routine command is what makes this differ from a generic if/else inside a recommendation function. The lockedcopy registry and the deterministic pipeline order are concrete mechanisms enforcing the preemption.

[0080] Mode-priority decision pipeline with safety-class preemption method further comprises the safety-class mode is alcohol-impairment, classified by a Widmark approximation, the safety-class command set includes at least one transportation safety command and at least one stop-intake command, and the standard generator's recheck cadence is itself overridable by a third source (the Autopilot of Hook 2), but the safety-class command is not BAC computation.

[0081] Sweat-derived sodium audit and intentional sodium-gap framing

[0082] Sweat-derived sodium audit and intentional sodium-gap framing is a method for computing, at the conclusion of a measured sweat session, both an estimate of sodium loss and an estimate of sodium delivered by the recommended brand intake, and for surfacing the difference as an intentional gap attributed to the brand's formulation strategy rather than as a deficit to be closed by additional sodium. Existing electrolyte-replacement guidance recommends matching sodium loss with sodium intake in a 1: 1 ratio. Marketed electrolyte products compete on sodium quantity (e.g. 1000 mg / serving). This positions sodium quantity as the primary axis of recovery quality, which is not consistent with cellular-uptake research the brand cites.

[0083] Sweat-derived sodium audit and intentional sodium-gap framing method comprises a sweat-rate engine that produces, in addition to a fluid-loss estimate, a sodium-loss estimate sodiumLossMg .

[0084] Sweat-derived sodium audit and intentional sodium-gap framing method comprises a brand-product sodium constant, BRAND_SODIUM_PER_UNIT_MG , the value of which is intentionally small relative to comparable marketed electrolyte products.

[0085] Sweat-derived sodium audit and intentional sodium-gap framing method comprises a prescription generator that sizes the number of brand servings recommended by fluid budget, not by sodium-matching, and that exposes the resulting brandSodiumTotalMg = servings × BRAND_SODIUM_PER_UNIT_MG.

[0086] Sweat-derived sodium audit and intentional sodium-gap framing method comprises a computation sodiumGapMg = max(0, sodiumLossMg – brandSodiumTotalMg) , surfaced to the user with framing that attributes the gap to non-sodium components of the brand formulation (the implementation uses "structured-water absorption + marine minerals").

[0087] Sweat-derived sodium audit and intentional sodium-gap framing method comprises a comparison-table UI that displays the brand sodium-per-serving alongside two or more comparable products' sodium-per-serving, anchored by a closer line stating that "more sodium is not always the goal."

[0088] Sweat-derived sodium audit and intentional sodium-gap framing method is implemented with a Sodium constant, Audit math, and an UI surface (Recovery Intelligence + Comparison Table).

[0089] The novelty is the inversion of the standard sodium-matching paradigm into an intentional gap that is calculated, surfaced, and explained as a product-design choice. The implementation

produces a numeric value (sodiumGapMg) and renders it in a way that the user can audit; this is different from marketing copy that simply asserts "less sodium is enough."

[0090] Sweat-derived sodium audit and intentional sodium-gap framing method further comprises BRAND_SODIUM_PER_UNIT_MG is approximately 25 mg, the prescription generator sizes brand servings by a per-serving fluid budget of approximately 12 oz, comparison-table UI is a card in a multi-card result pane that also includes the sodium-gap audit and a recovery-protocol recommendation gated by user-held inventory of the brand product.

[0091] In one embodiment, the Sweat-deficit-driven cadence override (Autopilot) consumes the live state defined in Pure-function performance scoring with audit-on-output and is consumed by the timer surface that Mode-priority decision pipeline with safety-class preemption mode-priority pipeline writes to.

[0092] In one embodiment, the Live-state beverage-replacement recommendation (Autoscan) consumes the same UserState snapshot type as Pure-function performance scoring with audit-on-output — the live-state property is what makes Autoscan's recommendation different from a static lookup.

[0093] In one embodiment, the Sweat-derived sodium audit and intentional sodium-gap framing is the user-visible payoff of the sweat session that Sweat-deficit-driven cadence override (Autopilot) uses to derive Autopilot.

[0094] System Implementation

[0095] As shown in **Fig. 4**, the System Implementation 400 includes a computer system or device 410, a mobile application 420, a cloud-based processing engine 430, and an optional wearable integration 440. The computing system or device 410 may be any computer system, such as a workstation, desktop computer, server, laptop, handheld computer, tablet computer (e.g., the iPad tablet computer), mobile computing or communication device (e.g., the iPhone™ communication device, Android™), or other form of computing or telecommunications device that is capable of communication and that has sufficient processor power and memory capacity to perform the operations described herein.

[0096] The computer systems 410 that may be used include, but are not limited to, mobile computing devices, such as wearable devices, cellular phones and personal digital assistants, and network equipment, such as load balancers, routers and switches. Other computer systems that may be used include, but are not limited to, personal computers, embedded computing systems,

network appliances, workstations, mainframes, networked clients, servers, media servers, application servers, database servers, web servers, virtual servers, and the like.

[0097] The computing device 410 includes one or more non-transitory computer-readable media for storing one or more computer-executable instructions or software for implementing exemplary
5 embodiments. The non-transitory computer-readable media may include, but are not limited to, one or more types of hardware memory, non-transitory tangible media (for example, one or more magnetic storage disks, one or more optical disks, one or more USB flashdrives), and the like. For example, memory 418 included in the computing device 410 may store computer-readable and computer-executable instructions or software for implementing exemplary embodiments.

[0098] The computer system 410 may include an operating system that manages at least a portion
10 of the hardware placements included in computer system 410. A processor or controller, such as processor 412, may execute an operating system which may be, among others, a Windows-based operating system (for example, Windows NT, Windows 2000/ME, Windows XP, Windows 7, or Windows Vista) available from the Microsoft Corporation, a MAC OS System X operating system
15 available from Apple Computer, one of many Linux-based operating system distributions (for example, the Enterprise Linux operating system available from Red Hat Inc.), a Solaris operating system available from Sun Microsystems, or a UNIX operating systems available from various sources. The operating system execute the mobile application 420 on a smart phone operating system, such as Windows Mobile, Android or iOS. Many other operating systems may be used,
20 and embodiments are not limited to any particular operating system.

[0099] A user may interact with the computing device 410 through an interface device 414, such
as a computer monitor, a touchscreen, or a visual display, which may display one or more user
interfaces that may be provided in accordance with exemplary embodiments. The visual display
may also display other aspects, elements and/or information or data associated with exemplary
25 embodiments, for example, views of databases, photos, and the like. The computing device 414
may include other input/output (I/O) devices for receiving input from a user, for example, a
keyboard or any suitable multi-point touch interface, a pointing device (e.g., a mouse), or camera.
The computing device 2200 may include other suitable conventional I/O peripherals.

[0100] The processor and operating system together define a computing platform for which
30 application programs in high-level programming languages may be written. These component
applications may be executable, intermediate (for example, C# or JAVA bytecode) or interpreted

code which communicate over a communication network (for example, the Internet) using a communication protocol (for example, TCP/IP). Similarly, functions in accord with aspects of the present invention may be implemented using an object-oriented programming language, such as SmallTalk, JAVA, C++, Ada, or C# (C-Sharp). Other object-oriented programming languages may also be used. Alternatively, procedural, scripting, or logical programming languages may be used.

[0101] Additionally, various functions in accord with aspects of the present invention may be implemented in a non-programmed environment (for example, documents created in HTML, XML or other format that, when viewed in a window of a browser program, render aspects of a graphical-user interface or perform other functions). Further, various embodiments in accord with aspects of the present invention may be implemented as programmed or non-programmed placements, or any combination thereof. For example, a web page may be implemented using HTML while a data object called from within the web page may be written in C++. Thus, the invention is not limited to a specific programming language and any suitable programming language could also be used.

[0102] A computer system included within an embodiment may perform functions outside the scope of the invention. For instance, aspects of the system may be implemented using an existing product. Aspects of the system may be implemented on database management systems such as SQL Server available from Microsoft of Seattle, Wash.; Oracle Database from Oracle of Redwood Shores, Calif.; and MySQL from Sun Microsystems of Santa Clara, Calif.; or integration software such as WebSphere middleware from IBM of Armonk, N.Y. However, a computer system running, for example, SQL Server may be able to support both aspects in accord with the present invention and databases for sundry applications not within the scope of the invention.

[0103] The mobile application (iOS/Android) may include various inputs including sensor data, manual logging, environmental APIs, or product scans (HydroScan). The mobile application (iOS/Android) may include various outputs including Performance score, Compliance-driven commands, Behavioral guidance. The mobile application (iOS/Android) may integrate products such as hydration formulations, Electrolyte + alkaline blends, or dosage tied to system output. The hydration formulations, Electrolyte + alkaline blends, or dosage tied to system output may be sent to the mobile application by a cloud processing engine 430.

[0104] Cloud-based processing engine 430 includes a network 432 suitable for a distributed implementation of exemplary embodiments. The network 432 may include one or more servers 434 coupled to one or more clients 434 via a communication network 2410. The network interface

440 and the network device 450 of the computing device 410 enable the servers 434 to communicate with the clients 434 via the communication network 432. The communication network 432 may include, but is not limited to, the Internet, an intranet, a Local Area Network (LAN), a Wide Area Network (WAN), a Metropolitan Area Network (MAN), a wireless network, an optical network, and the like. The communication facilities provided by the communication network 432 are capable of supporting distributed implementations of exemplary embodiments.

[0105] In an exemplary embodiment, the servers 434 may provide the clients 436 with computer-readable and/or computer-executable components or products under a particular condition, such as a license agreement. For example, the computer-readable and/or computer-executable components or products may include those for providing and rendering any of the user interfaces disclosed herein. The clients 436 may provide and render an exemplary graphical user interface using the computer-readable and/or computer-executable components and products provided by the servers 434.

[0106] Alternatively, in another exemplary embodiment, the clients 436 may provide the servers 434 with computer-readable and computer-executable components or products under a particular condition, such as a license agreement. For example, in an exemplary embodiment, the servers 434 may provide and render an exemplary graphical user interface using the computer-readable and/or computer-executable components and products provided by the clients 436. In one embodiment, the clients 436 provide the Deterministic AI Command Layer, AutoPilot Execution System, and/or the Closed-Loop Feedback System.

[0107] Optional wearable integration

[0108] The wearable device 440 comprises sensors 442 to detect physiological data, behavioral patterns, and environmental conditions. The wearable device 440 transmits physiological data, behavioral patterns, and environmental conditions to the computing device 410 via a network device 444. The wearable physiological measurement system may record and transmit at least the following types of data to an external computing system, mobile communication system, a cloud or non-cloud storage system, and/or the Internet: raw physiological data (e.g., heart rate data, hydration, movement data, galvanic skin response data) and processed data derived from the raw data (e.g., RR intervals determined from the heart rate data). Transmission modes may be wired (e.g., using USB stick inserted into a USB port on the system) or wireless (e.g., using a wireless transmitter). The raw and processed data may be transmitted together or separately using the same

or different transmission modes. Since a raw data file is typically substantially larger than a processed data file, in one non-limiting example, the raw data file may be transmitted using WiFi or a USB stick, while the processed data file may be transmitted using Bluetooth.

[0109] The network device 444 may include a 2G, 3G, 4G, 5G, or 6G chip that wirelessly uploads all data to the website disclosed herein without requiring any other external device. In some cases, the wearable system may opportunistically transfer data when in close proximity to a streaming outlet. For example, the system may avoid data transmission when it is not within close proximity of a streaming outlet, and, when nearby a streaming outlet (e.g., a linked phone), may send the data to the external device via Bluetooth and to the Internet via the external device. This is both convenient and “free” in the sense that the system utilizes existing cellular data plans.

[0110] Alternative embodiments

[0111] The system can be implemented using: wearables (bands, patches, smart devices), AI personalization models, Third-party integrations (Apple Health, Garmin, etc.), Voice assistants or alerts, or Enterprise/tea dashboards. The alternative modes and systems are coupled to the closed-loop command system through the cloud processing engine. The alternative modes and systems comprise autonomous commands, a closed-loop system, product integration, real-time execution, and structured AI output.

[0112] System Architecture

[0113] As shown in **FIG. 5**, the System Architecture 500 comprises the AForce OS mobile client 510, the api-server 530, the Postgres data store 550, and the WebSocket realtime channel 560 that synchronizes state across devices. The system architecture is a file-based routing through expo-router. Root Stack wraps a persistent bottom, Tabs layout for the main experience.

[0114] Table 1: Navigation Skeleton

Route	Purpose
/ (tab)	Hydration Control Center — animated status orb, AI command, prediction strip
/check (tab)	Performance Signals — biometric snapshot
/ protocol (tab)	Active recovery protocols
/ store (tab)	Product shopping (RTD, sticks, canister)
/ profile (tab)	User settings, HealthKit integration, Protocol Tools

/ scan (modal)	Camera-based product scanning (Autoscan)
/ heat	Heat-risk prediction dashboard
/ sweat	Sweat Intelligence v2 calculator (drives Autopilot)
/ competition	GPS-based territory map
/ subscription	Premium plan management
/(auth)/sign-in , /(auth)/sign-up	Clerk-backed auth

[0115] The Aforce OS mobile client 510 include the Home Orb 512 presenting the user score 0-100, a recovery card 514, a social mode User Interface 516, a voice overlay 518, and a Health Bridge 520 tracking heart rate variability (HRV), sleep, and Resting heart rate (RHR). The Aforce OS mobile client 510 operably communicates with the api-server 530, which comprises an authorization middleware 532, the score engine (pure, deterministic) 534, a rate limiter (per user) 536, a websocket hub 538, and a weather cache (10 min TTL) 540. The api-server 530 operably communicates with the Postgres data store 550, which the Postgres component 552, a clerk component 554, a stripe component 556, and an open weather component 558. The Postgres component 552 includes Drizzle ORM, User_state, history, entitlements. The clerk component 554 includes the auth JWT verification. The Stripe component 556 includes the checkout webhooks portal, and the open weather component 558 includes heat and humidity inputs. The api-server 530 operably communicates with the WebSocket realtime channel 560, which includes the state core entitlement events.

[0116] FIG. 6 the Mobile App Architecture 600 including a Presentation Layer (Screens and Section Components) 610, a State Layer 640, a I/O Layer (Mobile) 660, and a Cross-Cutting component 680. Sliced React Context + pure reducer + memoized slice projections so unrelated UI sections do not re-render on unrelated state changes. Expo-router file-based navigation. Clerk auth bridge mediates between Clerk session and the internal store. Apple HealthKit signals enter the engine through the appleHealth field of UserState. The Presentation Layer (Screens and Section Components) 610 includes the home screen orchestrator 612, the recovery card (locked/unlocked) 614, a social mode UI 616, voice overlay 618, onboarding module 620, settings component 622, homeheader/Orb Section 624, Command Stack/CTA 626, Entry Action/Signals 628, and section shared phone and foldable layouts 630. The state layer 620 includes an app store

reducer 642, an action creators/providers component 644, 9-slice projection 646, selector hooks 648, slices including engine, user, social subscription, flags, intake, cycle, confirmation, and onboarding 650, and persistence including async storage rehydrate on cold start, write through on every action 652. The I/O layer (mobile) 660 includes a REST client 662, Apple Health 664, Push Notifications 666, WebSocket Client 670, and Voice I/O 668.

[0117] Table 2: State Management

Layer	Responsibility
Store orchestrator	Owns AppState, dispatch, periodic 30s server polling, WebSocket subscription
Reducer (pure)	All state transitions: intake logging, timer ticks, autopilot, engine refreshes
Slice projections	EngineSlice, UserSlice, SocialSlice, CycleSlice, InventorySlice, SweatAutopilotSlice
Type contract	AppState, UserState, ScoreEngineOutput, IntakeEvent, InventoryState , SweatAutopilot
Layer	Responsibility

[0118] Table 3: Hardware integrations

Surface	Notes
Voice	Classifies transcripts into intents (LOG_INTAKE , GET_STATUS , etc.) and renders brand-aligned spoken responses
Camera	expo-camera CameraView reading ean13 , upc_a , qr , code128
Apple HealthKit	Apple HealthKit
Auth	Bridges Clerk session → internal store; (tabs) layout redirects unauthenticated users
Entitlement	Feature gating (subscriber vs. free)

[0119] As shown in **FIG. 7**, the Wireframes 700 including the five primary tabs of AForce OS — Hydration Control Center (home), Performance Signals (check), Recovery Protocol, Store, and Profile — plus the Sweat Calculator and Autoscan modal entry points. All wireframes are also available as live, interactive iframes on the engineering canvas. The wireframes 700 include the Full live AForce OS app 702, Recovery Card — subscriber view 704, Recovery Card — paywall 706, and Hydration Score Band component 708

[0120] As shown in **FIG. 8**, the Scoring Engine 720 including the pure-function pipeline from UserState snapshot through (a) per-event absorption, (b) bounded contribution sum, (c) continuous decay model, to (d) score / band / breakdown / prediction / command. Server- and client-side execution produce identical output for identical input ("score identity invariant"). The Scoring Engine is a pure function that takes a UserState snapshot and returns a ScoreEngineOutput. It runs both server-side (during state writes) and client-side (on every API response, for sub-network latency UI updates). Because it is pure, the same input snapshot always produces the same output — this is the foundation of the determinism, as described below.

[0121] Scoring Engine 720 includes Inputs (UserState + environmental snapshot) 722, Deterministic Computation Pipeline 740, Score aggregator 762, performance state classifier 764, and Outputs (Score Engine Output) 770. The inputs 722 include intake Events, recent logs with pre-computed absorption impact; ounces (Oz) Consumed Today, base hydration progress 724; aforce Units Today — count of AForce-specific intakes (drives the bonus term); last Intake Time 738 timestamp for decay and recovery-momentum calculations; heat Load 730, sweat Rate 728, environmental + physiological stressors (0–10); urine Signal — biomarker proxy (1–8 scale); symptom State 726, symptoms[] — severity bucket + specific symptom IDs; apple Health 732 — hrv Sdnn , sleep Hours Last Night from HealthKit; clutchActive — boolean; multiplies decay during high-intensity windows; social Mode 734— alcohol mitigation state (drinks, ABV, food)

[0122] The Outputs (Score Engine Output) 770 include the Score (0-100) 772, Performance State Level+color+urgency 774, pulse Config animation/glow/wave 776, Command Next action + impact 778, RiskTimer Recheck minutes 780, Breakdown [] Per factor contributions 784, Prediction Decay/min +minutes to depleted 786, Social (rollup or null) 788, No I/O, no logging, no clock reads outside fn args 790.

[0123] Score formula = base_intake + aforce_bonus + recency + consistency + context + recovery_momentum + symptom_penalty (negative) + urine_penalty (negative) + output_stress

(negative) + sleep_carry + recovery_signal + confirmation. Each term is a bounded contribution with its own clamp and weight, exposed in the breakdown array so the UI can render an audit (the "Why this score?" sheet in components/).

[0124] Absorption model (hydrationScoreService.ts:25) comprises Plain water: 60 % immediate, 40 % over 12.5 min. AForce: 70 % immediate, 30 % over 25 min (faster onset because of structured-water + mineral pairing). Cap: at most 1.5 units per rolling 20-min window; excess is 75 % efficient (models gut absorption ceiling)

[0125] Continuous decay (scoringEngine.ts:166–202) comprises decay is applied per-minute and surfaces in the UI as the prediction string ("Drops to DEPLETED in 1 h 12 m").

[0126] Table 4: Score bands (scoringEngine.ts:38–42)

Band	Range	Default recheck interval (scoringEngine.ts:444–447)
PEAK	90 – 100	≥ 20 min
BALANCED	75 – 89	≥ 15 min
RECOVERING	60 – 74	≥ 10 min
DEPLETED	0 – 59	≥ 5 min

[0127] These bands are overridden by the Autopilot system when an active sweat session is in progress.

[0128] UI consumption is driven exclusively by(store/slices.tsx) Primary consumers: components/StatusPulseOrb.tsx — uses for the animated orb, components/AnimatedScore.tsx—counts the score up/down, components/ScoreBreakdownSheet.tsx—renders breakdown[], components/home/OrbSection.tsx — renders command.action +, Home screen "Prediction Strip" — renders prediction.label.

[0129] **FIG. 9** shows the Deterministic AI Layer is an ordered, pure-function rule pipeline with no remote inference and no nondeterministic LLM calls in the safety- and performance-critical path. The Deterministic AI layer comprises an ordered rule pipeline: mode classification → penalty application → bonus application → decay projection → command generation → recheck cadence. No remote LLM inference in the safety- and performance-critical path; the same snapshot always produces the same command. Safetyclass commands (Social Mode, Guardian Risk Engine) preempt the standard band ladder. The deterministic AI layer 800 includes Stage 1 Signal Fusion

802, Stage 2 – Normalization and Confidence 816, Stage 3 – Rule Pipeline (deterministic, ordered, explainable) 826, Stage 4 – Decision Synthesis 840.

[0130] Conventional consumer health apps make recommendations through probabilistic LLM calls. The AForce Deterministic AI Layer is the opposite: an ordered, pure-function rule pipeline with no remote inference and no nondeterministic LLM calls in the safety- and performance-critical path. The same snapshot always produces the same AICommand. This matters for three reasons that the application leans on as differentiators, Auditability, Safety-class behavior, and Edge-first. Auditability. Every output can be reproduced offline by replaying the snapshot through the same code. The array exposes every contributing term. Safety-class behavior. Alcohol-mitigation recommendations ("DO NOT DRIVE", "PULL FROM ROTATION") cannot be subject to model drift or hallucination. Edge-first. Decisions are computed locally on the device (or in the same reducer function on the server). The app degrades gracefully offline.

[0131] Stage 1 Signal Fusion 802 includes the Biometric Stream including HRV, RHR, sleep, steps 804, Intake Stream including logged drinks/sticks 806, Symptom stream including Urine, energy, self-report 808, Environmental including Weather, time of day 810, Behavioral including Streak, compliance 812, and Social including BAC inputs 814.

[0132] Stage 2 – Normalization and Confidence 816 includes Unit normalization including Oz/mL/Celsius/Fahrenheit 818, Range clamps & validators including Reject implausible values 820, Per-signal confidence including High/med/low 822, and Freshness check including Stale data down weighted 824.

[0133] Stage 3 – Rule Pipeline (deterministic, ordered, explainable) 826 includes Hydration Rules 828, Heat/sweat rules 830, Recovery Rules HRV-aware 832, Social/ BAC rules 834, Snooze/quiet hours 836, and Each rule emits: contribution to score, reason string, urgency tag, candidate command. Rules are pure functions 838.

[0134] Stage 4 – Decision Synthesis 840 includes Score aggregator including Weighted sum 842, Command selector including Highest impact, urgency-tied 844, Reasoning trace including Reasons array surface in UI 846, Pulse config builder including Animation maps to urgency 848, and Auditable: every score and command can be reproduced from a captured input snapshot. No remote inference. No nondeterministic LLM calls in this layer 849.

[0135] Rule pipeline (Stage 3)

[0136] The pipeline (**FIG. 9**) runs in this order, top-down, on each engine refresh:

[0137] 1. Mode classification — Is the user in a Social Mode override (alcohol present)? If yes, skip the standard ladder; the Social Mode rule set takes precedence (scoringEngine.ts:458–531). Otherwise, use the standard PEAK / BALANCED / RECOVERING / DEPLETED ladder from Table 4.

5 [0138] 2. Penalty application — Symptom severity, urine-color signal, output stress, and HRV/sleep deficits each apply a bounded negative delta.

[0139] 3. Bonus application — AForce intakes, consistency streaks, recovery momentum, and confirmed intakes each apply a bounded positive delta.

10 [0140] 4. Decay projection — Continuous decay model yields prediction. Minutes To Depleted and the prediction label.

[0141] 5. Command generation — generateCommand() (lines 537–591) selects a "WHAT + WHEN + OUTCOME" directive based on state, deficit, and inventory.

[0142] 6. Recheck cadence — Default cadence by band; if Autopilot is active, replaced by the autopilot interval (Cadence layer.

15 [0143] **Social Mode override**

[0144] **FIG. 10** shows the Social Mode Flow 850 including the Drink-logging panel → Widmark BAC estimate (parameterized by sex, weight, drinks, ABV, time elapsed, food intake) → impairment-level mapping (LOW / ELEVATED / MODERATE / HIGH / CRITICAL) → safety-class command override that preempts the standard performance ladder.

20 [0145] When social Mode is active, the engine swaps in a parallel ruleset that: 1) Estimates BAC via the Widmark approximation (services/bacEstimationService.ts:92–147), parameterized by sex, weight, drinks consumed, ABV, time elapsed, and food intake; 2) Maps BAC to one of five impairment levels — LOW, ELEVATED, MODERATE, HIGH; and 3) Issues escalating commands and surfaces transportation / legal safety language. The disclaimer copy is locked and
25 globally safe.

[0146] The Social Mode 850 includes the user 852 which then Activates Social Mode 854 including the Sex, ate-recently, started-at. User Logs Drink 856 including type, oz, ABV. User can Deactivates Social mode 858 where the recovery window starts and Recovery Window (server-clocked) 876 where Social rollup persists, then clears. Or the user can
30 POST/api/aforce/social/drink 860. Then the Score Engine: Social Rollup 862 determines BAC, alcohol multiplier, impairment, which can engineOutput, social attached broadcast via WS 864; or

determine a Recovery threshold 866. If no recovery threshold is reached, then Continue logging 868. If recovery threshold is reached then, Recovery banner Shown in Signals zone 870. if impairment =elevated, then Transportation prompt 872 and a Rideshare deep-link is provided. BAC, impairment, and transportation prompts are all directed from the same deterministic Score Engine – client never computes these locally.

[0147] Guardian Risk Engine and Clutch Strategy

[0148] Two specialized branches sit on top of the core pipeline:

[0149] 1. Guardian Risk Engine (scoringEngine.ts:728–748) — applies an additional risk overlay when symptoms + heat + low HRV co-occur, escalating the command urgency and tightening the recheck cadence.

[0150] 2. Clutch Strategy (scoringEngine.ts:790–838) — when is set (user has marked a high-intensity competitive window), the engine multiplies decay by 1.3, prefers AForce-led commands, and shortens cadence.

[0152] Outputs include: command.action — short imperative ("16 oz + 2 sticks now"); command.explanation — rationale shown beneath the command; command.urgencyLevel — low | medium | high | critical; command.estimatedImpact — predicted score delta if executed; pulseConfig — drives the orb animation so the user can read state at a glance; recheckMinutes — fed to the timer (overridden by Autopilot when applicable).

[0153] Autopilot

[0154] **FIG. 11** shows Autopilot 900 / Autoscan 940, where Autopilot 900 is a physiologically-paced recheck cadence that takes over the timer for a fixed window (4 h) following a measured sweat session. Instead of using the band-default cadence from, the timer is driven by the user's just-measured hydration deficit % so high-deficit athletes get checked on more aggressively for the duration of the recovery window. Top half: sweat session → deficit % → autopilot cadence ladder ($\geq 4\%$ → 8 min / critical, $\geq 2\%$ → 12 min / high, else → 20 min / moderate, 4 h window) → atomic state transition that simultaneously stores the autopilot tuple, resets the visible recheck timer, and clears any pending confirmation prompt. Bottom half: barcode / QR scan → product recognition (local catalog → Open Food Facts fallback) → live-state fit score → conditional AForce replacement decision when brand fit exceeds scanned fit by > 4 points.

[0155] Autopilot 900 includes Wake/cold-start 902, Hydrate store from disk including AsyncStorage rehydrate 904, Open WS, auth via JWT 906, GET/api/aforce/state including Baseline snapshot 908, Apply SET_USER_STATE 910 including Engine output adopted, and Recheck timer armed 912 including Rom risk Timer minutes.

- 5 [0156] Trigger: The user runs the Sweat Calculator in any of three modes (Quick / Precision / Estimate) and taps Calculate. `commitSession()` (`SweatCalculatorScreen.tsx:167–172`) dispatches `SET_SWEAT_AUTOPILOT` with the session's autopilot derivation.

[0157] Table 5: Cadence ladder

Deficit	Interval	Urgency	Window
$\geq 4\%$	8 min	critical	4 h
$\geq 2\%$	12 min	high	4 h
$< 2\%$	20 min	moderate	4 h

- 10 [0158] **Reducer wiring**

[0159] `SET_SWEAT_AUTOPILOT` does three things atomically:

[0160] 1. Stores `sweatAutopilot` (interval + urgency) and `sweatAutopilotSetAt` (timestamp, used to enforce the 4 h window).

- 15 [0161] 2. Resets `state.timerSeconds = autopilot.intervalMin × 60` so the visible countdown immediately reflects the new cadence.

[0162] 3. Clears any pendingConfirmation so the new cadence is not blocked by a stale prompt.

[0163] **Consumption**

[0164] 1) if (`autopilotSession` is active within 4h) `recheckIntervalMin = autopilot.intervalMin`, `recheckUrgency = autopilot.urgency`, and `autopilotActive = true`.

- 20 [0165] 2) else `recheckIntervalMin = heatBandDefault`, `recheckUrgency = heatBandUrgency`, `autopilotActive = false`.

[0166] 3) `Components/RiskTimerDisplay.tsx` renders the resulting `timerSeconds` and shows an "URGENT" badge when state is DEPLETED or remaining time is < 5 min.

[0167] **Hand-off and graceful exit**

- 25 [0168] After 4 h, `useHeatGuard` falls back to band-default cadence — there is no need to explicitly "exit" Autopilot. The 4 h horizon is the Sawka 2007 post-exercise rehydration window and was

chosen so the cadence aligns with the period during which sweat-driven sodium and fluid replacement is physiologically meaningful.

[0169] Sodium positioning carried through Autopilot

[0170] The Sweat Calculator that triggers Autopilot also feeds the Recovery Intelligence card with two derived values that anchor AForce's "25 mg sodium is intentional" positioning: 1) $\text{aforceSodiumTotalMg} = \text{aforceSticks} \times \text{AFORCE_SODIUM_PER_UNIT_MG} (=25)$ — $\text{sweatRateEngine.ts:128,509}$; 2) $\text{sodiumGapMg} = \max(0, \text{sodiumLossMg} - \text{aforceSodiumTotalMg})$ — the intentional gap, framed as "covered by structured-water absorption + marine minerals".

[0171] These values are visible in the user-facing audit copy ("125 mg delivered by 5 units; gap 1 723 mg covered by structured-water absorption") and are the basis of the comparison-table claims (AForce 25 mg / Gatorade ~270 mg / LMNT 1 000 mg / Liquid IV ~500 mg).

[0172] Autoscan

[0173] Autoscan is a "scan-to-decide" pipeline. The user points the phone at any beverage barcode (or types one in). The system identifies the product, scores it against the user's current physiological state using the same engine described previously, and — if an AForce SKU would outperform the scanned product — issues a one-tap replacement recommendation.

[0174] As shown in FIG. 11, Autoscan 940 includes a Continuous Loop (every tick) with a TICK_Timer 916 Every 1 s while awake. If the Timer 914 and seconds do not = 0, then it keeps ticking and if the seconds =0, then Pending Confirmation=true 918 and Prompts the user to recheck. Confirm_Command 920 and engine recompute. Polling: Apple Health snapshot 922 is Periodic +on foreground, which SET_APPLE_HEALTH 924 and Attaches snapshot, then Engine re-runs 926 and New score broadcast. Polling: weather refresh 928 which Lat/Lon is cached endpoint, then userState.weather* 930 is updated, then Engine re-runs 932. The Server WS broadcast 934 that State, score, entitlement details, then Reducer applies updates 936 and No client side recompute, then UI reflect automatically 938. Loop is server clocked. Client never invents score deltas. It only renders broadcast state.

[0175] Table 6: Input modes

Mode	Used on	Mechanism
------	---------	-----------

Native barcode / QR	iOS, Android	expo-camera CameraView (CameraScanModal.tsx:38–48) reading ean13 , upc_a , qr , code128 . AForce-specific QR codes use the scheme aforce://product/<id>
Manual search	All platforms	Free-text product / brand / digit lookup
Mock scan tray	Web preview, demo, testing	HydrationScanScreen.tsx:217–250 exposes a fixture tray of barcodes for one-tap simulation

[0176] Resolution pipeline (services/productRecognitionService.ts)

[0177] 1. Look the barcode up in the local BARCODE_INDEX (data/productDatabase.ts) — instant, offline, zero network.

5 **[0178]** 2. If miss, fall back to the Open Food Facts API (productRecognitionService.ts:126) so any US beverage can be resolved even when not in the local catalog.

[0179] 3. Normalize the resolved product into the local ScannedProduct schema (name, brand, category, hydration markers, electrolyte density, sugar level, mineral content).

[0180] Scoring (services/hydrationScanService.ts:55–63)

10 **[0181]** The product is scored on: Hydration efficiency (0–100 %) — mineral content + water content – sugar penalty; Fit score (0–100) and verdict (Optimal / Strong / Acceptable / Suboptimal / Avoid) based on the user's current protocol context (Heat Stress, Recovery, Social, etc.) — i.e. the same UserState that drives Scoring Engine and the Deterministic AI layer.

[0182] 5.5 AForce comparison (hydrationScanService.ts:70 , 93–101)

15 **[0183]** The catalog is filtered to isAForce: true SKUs. The engine picks the best AForce product for the user's current state, computes its fit score, and compares to the scanned product. If $aforceFitScore \geq scannedFitScore + 4$, an AForce Replacement Card (components/AForceReplacementCard.tsx) is rendered on the result, including the situational headline, detail, and a specific command ("Take 1 AForce Stick now with 16 oz water").

20 **[0184]** Output (ScanOutcome) includes product — resolved details; efficiency — 0–100 % hydration efficiency; fitScore + verdict — for the scanned product; aforceUplift — if applicable: {aforceProduct, fitDelta, command}; recommendation — situational headline + detail + command, rendered by components/ScanResultCard.tsx

[0185] Autoscan novelty

[0186] Existing barcode-scanning beverage apps return static nutrition facts. Autoscan scores a product against a live user state derived from the same Deterministic AI Layer that drives the rest of the app and replaces a static "facts" output with a contextual decision (drink it / don't / use AForce instead). The replacement decision is auditable because both fit scores are computed by the same pure scoring code.

[0187] Glossary

[0188] AForce OS. The mobile application built in artifacts/aforce-os/. Expo / React Native, served at / in development and to be published as the AForce iOS / Android app. The "OS" in the name refers to the role the app plays for the user — a continuous performance operating system, not a logging journal.

[0189] AForce serving / unit. One physical AForce dose, defined as 12 oz of finished beverage. May be delivered as a Ready-to-Drink (RTD) bottle, a stick mix dissolved in water, or a scoop from a canister. The engine treats all three formats as the same unit for purposes of aforceUnitsToday and the absorption-cap math.

[0190] AFORCE_SODIUM_PER_UNIT_MG. Constant exported from services/sweatRateEngine.ts:128, value 25. Each AForce serving contributes 25 mg of sodium. The intentionally small number is the foundation of the "recovery is not driven by sodium alone" positioning surfaced on the Recovery Intelligence card.

[0191] AICommand. Output object produced by generateCommand() (utils/scoringEngine.ts:537). Contains action (short imperative), explanation (rationale), urgencyLevel (low | medium | high | critical), and estimatedImpact (predicted score delta).

[0192] AppState. Top-level shape held by useAppStore and mutated only via the pure reducer in store/appStoreReducer.ts. Includes userState, engineOutput, timerSeconds, pendingConfirmation, sweatAutopilot, sweatAutopilotSetAt, history and the boolean UI flags.

[0193] Apple Health / HealthKit. iOS biometric system. The bridge in services/appleHealth.ts reads restingHeartRate, hrvSdnn, and sleepHoursLastNight. The values flow into UserState.appleHealth and contribute to the score via the HealthKit branch in buildBreakdown().

[0194] Autopilot. A physiologically-paced recheck cadence triggered by completing a Sweat Calculator session. Replaces the band-default recheck timer for a fixed 4-hour window with an interval derived from the user's just-measured hydration deficit.

- [0195] Autoscan / HydroScan. The product-recognition pipeline (screens/HydrationScanScreen.tsx, services/hydrationScanService.ts). User points the phone at a barcode; the app identifies the product, scores it against the live user state, and conditionally suggests an AForce replacement.
- 5 [0196] BAC. Blood alcohol concentration. Estimated by services/bacEstimationService.ts:92 using the Widmark approximation, parameterized by sex, weight, drinks, ABV, time elapsed, and food intake. Drives the Social Mode override in the Deterministic AI Layer.
- [0197] Band (performance band). One of PEAK / BALANCED / RECOVERING / DEPLETED, determined by resolveState() (utils/scoringEngine.ts:38) at the thresholds 90 / 75 / 60. Each band
- 10 has a default recheck interval and a default pulse animation. (
- [0198] BaseDecay. Per-minute score-decay rate computed by computeDecayPerMinute() (utils/scoringEngine.ts:166). Function of body weight, activity level, ambient temperature, humidity, sleep clutch state, and any active alcohol multiplier.
- [0199] Breakdown. Array of ScoreContribution objects ({id, label, delta, maxMagnitude, hint})
- 15 returned alongside every score . Powers the "Why this score?" sheet (components/ScoreBreakdownSheet.tsx) — every score shipped to the UI is auditable.
- [0200] Cadence ladder (Autopilot). The 8 / 12 / 20 minute interval table returned by deriveAutopilot(deficitPct) (services/sweatRateEngine.ts:489).
- [0201] Catalog (product). Local beverage database in data/productDatabase.ts indexed by barcode
- 20 and product ID. Supplemented at runtime by Open Food Facts for unknown barcodes.
- [0202] Clerk. Third-party auth provider integrated via ClerkProvider (root layout) and components/ClerkAuthBridge.tsx. The (tabs) layout redirects unauthenticated users to /(auth)/sign-in.
- [0203] Clutch Strategy. Performance overlay in utils/scoringEngine.ts:790. When clutchActive is
- 25 true (user marked a high-intensity competitive window), decay is multiplied by 1.3, AForce-led commands are preferred, and the recheck cadence is shortened.
- [0204] Command (AICommand). See AICommand.
- [0205] Compliance streak. userState.complianceStreak — number of consecutive days the user has hit their oz target. Adds up to +15 to the score via the consistency term.

[0206] Decay. The per-minute score reduction applied between intakes. See `aseDecay`. The integrated total over `[lastIntakeTime, now]` is computed by `computeDecayPoints()` (`utils/scoringEngine.ts:215`).

[0207] Deficit % (`deficitPct`). Hydration deficit expressed as a percentage of body mass, computed by the sweat-rate engine. The single input that drives the Autopilot ladder.

[0208] DEPLETED. Lowest performance band, score < 60. Triggers the `cmd-depleted` command and a 5-minute recheck interval by default.

[0209] Deterministic AI Layer. The complete rule pipeline that takes a `UserState` snapshot and returns an `AICommand`. Pure-function, no remote inference, identical input $\Rightarrow$ identical output.

[0210] `EngineSlice`. Memoized React context (`store/slices.tsx`) projecting `engineOutput` so consumers re-render only when the score / command actually changes.

[0211] `expo-camera`. Native camera library used for barcode scanning in `components/CameraScanModal.tsx`. Reads `ean13`, `upc_a`, `qr`, and `code128` symbologies.

[0212] `expo-router`. File-based navigation library used by app/. Routes derive from filesystem paths (e.g. `app/(tabs)/index.tsx` $\rightarrow$ /).

[0213] Fit score. 0–100 score produced by `services/comparisonEngine.ts` for any beverage against a given user state. Used by Autoscan to decide whether to recommend an AForce replacement.

[0214] Guardian Risk Engine. Composite risk score computed by `guardianRiskScore()` (`utils/scoringEngine.ts:728`) from hydration %, body weight, active minutes, heat index, sweat rate, core temp estimate, quarter, and pH. Tiered into OPTIMAL / WATCH / MODERATE / CRITICAL by `guardianTier()`.

[0215] Heat Guard. Hook (`hooks/useHeatGuard.ts`) that combines climate input + symptom load + Apple Health into a heat-stress score and a recheck cadence. The primary consumer of Autopilot — when an autopilot window is active, Heat Guard returns the autopilot interval instead of its own band default.

[0216] HRV (`hrvSdnn`). Heart-rate variability, SDNN method, in milliseconds. Read from Apple Health and used as a recovery-quality signal in the score formula.

[0217] HydroScan. See Autoscan.

[0218] Impairment level. One of LOW / ELEVATED / MODERATE / HIGH / CRITICAL, mapped from BAC by `services/legalSafetyService.ts:28`. Determines which Social Mode command the engine emits.

- [0219] IntakeEvent. Single logged consumption event ({loggedAt, oz, fluidType, flavor, ...impact}). The list userState.intakeEvents drives the per-event absorption math.
- [0220] Inventory. userState.inventory — {sticks, rtd, canister} integer counts. Used by the Sweat Calculator's Recovery Protocol resolver (RTD > sticks > canister, restock fallback).
- 5 [0221] Open Food Facts. Free public beverage / food database. Used by services/productRecognitionService.ts:126 as a fallback when a barcode is not in the local catalog, so any US beverage can resolve to a ScannedProduct.
- [0222] PEAK. Top performance band, score ≥ 90 . Triggers the cmd-peak command and a 20-minute recheck interval by default.
- 10 [0223] pulseConfig. Animation configuration for the StatusPulseOrb component. Fields include waveBehavior and animations. Computed in the engine and passed verbatim to the orb.
- [0224] Pure function. A function whose output is determined solely by its inputs and which has no observable side effects. Both the score engine and the reducer are pure; this is the foundation of the determinism claim.
- 15 [0225] RECOVERING. Mid-low performance band, $60 \leq \text{score} < 75$. Triggers the cmd-recovering command and a 10-minute recheck interval by default.
- [0226] Recheck interval. Time, in minutes, until the timer expires and the user is prompted to log again. Determined by band default unless an Autopilot window is active.
- [0227] Recovery window. The 4-hour period after a sweat session during which Autopilot drives
20 the recheck cadence. Length follows the Sawka 2007 post-exercise rehydration window.
- [0228] RTD. Ready-to-drink — a pre-mixed AForce bottle (11 oz / 325 ml). One of three product formats; preferred by the Sweat Calculator's protocol resolver when in stock. (
- [0229] Sawka 2007. Reference to Sawka et al., "Exercise and Fluid Replacement," ACSM Position Stand, 2007.
- 25 [0230] Source for the sweat formula (pre – post + fluid – urine) / duration and the 4-hour post-exercise rehydration window.
- [0231] ScoreEngineOutput. Top-level engine return shape.
- [0232] ScoreContribution. Single line in the breakdown — {id, label, delta, maxMagnitude, hint}. Bounded by maxMagnitude so any one term cannot dominate.

[0233] SET_SWEAT_AUTOPILOT. Reducer action dispatched by commitSession() after a Sweat Calculator run. Stores sweatAutopilot + sweatAutopilotSetAt, resets timerSeconds = intervalMin × 60, and clears pendingConfirmation.

[0234] Slice. Memoized React context that projects part of AppState. Slices: EngineSlice, UserSlice, SocialSlice, CycleSlice, InventorySlice, SweatAutopilotSlice.

[0235] Social Mode. The alcohol-mitigation path. When userState.socialMode.active is true, the Deterministic AI Layer swaps in a parallel ruleset (BAC, impairment, transportation safety) that takes precedence over the standard performance ladder.

[0236] Sodium gap (sodiumGapMg). max(0, sodiumLossMg - aforceSodiumTotalMg) — the intentional shortfall surfaced as "covered by structured-water absorption + marine minerals" on the Recovery Intelligence card.

[0237] Sweat Calculator (Sweat Intelligence v2). Screen at /sweat (screens/SweatCalculatorScreen.tsx). Three input modes — Quick / Precision / Estimate — feed services/sweatRateEngine.ts, which produces sweat rate, deficit %, sodium loss, AForce sodium delivered, sodium gap, and the Autopilot derivation.

[0238] SweatAutopilot. Type returned by deriveAutopilot(). Shape: {intervalMin, urgency, recoveryWindowHours}.

[0239] Urine signal. userState.urineSignal on a 1–8 scale. 1 = clear; 8 = dark amber. Concentration penalty engages at ≥ 5; clear-urine bonus engages at ≤ 2. Sourced from the user's color-card self-report in the check tab.

[0240] UserState. Single Postgres-backed user-record snapshot. Inputs to every engine call; mutated only through reducer actions and server writes.

[0241] Urine signal. userState.urineSignal on a 1–8 scale. 1 = clear; 8 = dark amber. Concentration penalty engages at ≥ 5; clear-urine bonus engages at ≤ 2. Sourced from the user's color-card self-report in the check tab.

[0242] UserState. Single Postgres-backed user-record snapshot. Inputs to every engine call; mutated only through reducer actions and server writes.

[0243] Voice service. services/voiceService.ts orchestrator. Classifies a transcript into an intent (LOG_INTAKE, GET_STATUS, ...), executes it, and renders a brand-aligned spoken response via voiceTemplateEngine.ts.

[0244] WebSocket subscription. services/realApi.ts#subscribeToStateUpdates. Real-time push channel from apiserver ; lets a second device (e.g. an Apple Watch in the future, or a coach's phone today) reflect a state update within a second of it being persisted.

5 [0245] Widmark approximation. The standard pharmacokinetic BAC equation, parameterized by sex (r factor), body weight, alcohol mass consumed, and elimination rate. Implemented in services/bacEstimationService.ts:92.

[0246] Table 7: Common acronym

Acronym	Expansion
ABV	Alcohol By Volume
ACSM	American College of Sports Medicine
BAC	Blood Alcohol Concentration
BSA	Body Surface Area
HRV	Heart Rate Variability
OFF	Open Food Facts
RTD	Ready-To-Drink
SDNN	Standard Deviation of NN intervals (HRV method)
WS	WebSocket

[0247] Examples

10 [0248] The following examples are put forth so as to provide those of ordinary skill in the art with a complete disclosure and description of how the compounds, compositions, articles, devices and/or methods claimed herein are made and evaluated, and are intended to be purely exemplary of the invention and are not intended to limit the scope of what the inventors regard as their invention. However, those of skill in the art should, in light of the present disclosure, appreciate
15 that many changes can be made in the specific embodiments which are disclosed and still obtain a like or similar result without departing from the spirit and scope of the invention.

[0249] Efforts have been made to ensure accuracy with respect to numbers (e.g., amounts, temperature, etc.), but some errors and deviations should be accounted for. Unless indicated
20 otherwise, parts are parts by weight, temperature is in ° C. or is at ambient temperature, and pressure is at or near atmospheric.

[0250] Example 1: SWEAT QUANTIFICATION & INVENTORY-GATED HYDRATION PRESCRIPTION ENGINE

[0251] The invention relates to a software system that quantifies an athlete's per-session sweat and electrolyte loss, classifies the resulting physiological state, and produces a closed-loop hydration prescription that is gated by the user's actual on-hand product inventory and that intentionally leaves a residual sodium gap to favor a specified absorption pathway. The system further provides three computationally distinct user-input modes that gracefully degrade based on the equipment available to the user (clinical scale, simple scale, or no scale), while preserving an invariant output schema.

[0252] Problem to be solved

[0253] Existing sweat-rate calculators in the consumer fitness space fall into one of two camps:

[0254] Lab-only / clinical — Require pre/post weigh-in on a calibrated scale, a urine-output measurement, and trained interpretation. Inaccessible to ~99% of recreational and competitive athletes.

[0255] One-shot consumer tools — Single fixed input form, no fallback when data is missing, no environmental adjustment, no acclimatization handling, no sodium classification, no recommendation tied to a specific product, and no recheck loop.

[0256] Neither addresses the central commercial problem: a hydration product manufacturer needs a calculator whose output is operationally actionable through the manufacturer's own SKU lineup, with a recommendation that is honest about what the product can deliver and what it cannot, and that adapts to whatever measurement equipment the user happens to have.

[0257] Tri-Modal Graceful Degradation with Invariant Output Schema

[0258] The engine accepts user input under one of three modes —Quick, Precision, Estimate — selected based on what equipment the user has available:

[0259] Table 8

Mode	Required equipment	Required inputs	Optional inputs
Quick	Body scale	pre-weight, post-weight, duration, fluid intake	height (improves BSA scaling)
Precision	Clinical-grade scale + urine collection	Quick fields + urine loss + sport + ambient temp +	—

		humidity + acclimatization flag + sodium profile	
Estimate	None (no scale)	body weight, height, sport, intensity (1–5), ambient temp, humidity	acclimatization, sodium profile

[0260] The novelty is not the modes themselves but the invariant output contract: regardless of which mode is invoked, the engine produces an identical SweatSession object containing sweatLossL , sweatRateLh , deficitPct , deficitBand , sodiumLossMg , a prescription sub-object, and an autopilot recheck profile. Downstream UI, history storage, and recovery protocol systems can therefore consume any mode’s output uniformly. Mode-selection is presented as a graceful capability ladder rather than three disconnected tools.

[0261] A method for computing per-session sweat loss in which (a) at least three distinct input forms are accepted, (b) each input form is mapped through a different physiological model (measured for Quick/Precision; BSA-anchored empirical for Estimate), and (c) all three forms emit a structurally identical output object enabling downstream consumers to remain mode-agnostic.

[0262] Dual-Direction Heat-Acclimatization Coupling

[0263] When the user is flagged as heat-acclimatized, the engine simultaneously applies two adjustments in opposite directions:

[0264] Sweat rate is multiplied by 1.08 (acclimatized athletes sweat ~8% more in volume — earlier onset, higher peak)

[0265] Sweat sodium concentration is multiplied by 0.7 (acclimatized athletes lose 30% less sodium per liter— physiological adaptation of the sweat gland)

[0266] This dual-direction coupling — increasing volume estimate while decreasing concentration estimate from a single boolean input — produces a substantially different total sodium prediction than either adjustment alone. To our knowledge, no consumer sweat-rate tool implements both directions simultaneously from a single user signal, and most omit acclimatization entirely.

[0267] Inventive claim element: A method wherein a single binary user input (“acclimatized: true|false”) triggers a coupled pair of multiplicative adjustments — one that increases predicted

sweat volume and one that decreases predicted sweat sodium concentration — applied within the same calculation pass.

[0268] Inventory-Gated Prescription with Intentional Sodium Gap

[0269] This is the most commercially distinctive element of the engine.

5 **[0270]** After the engine computes total sodium loss in milligrams, it does NOT recommend the dosage required to neutralize the loss. Instead it executes a constrained allocation:

[0271] 1) Inventory preference order: RTD (ready-to-drink) → Stick packs → Canister scoops. The system queries the user’s recorded on-hand product state to pick the available form factor.

10 **[0272]** 2) Allocation rule: One stick or scoop per 12 oz of replacement water. This is a fixed product-tied ratio rather than a variable computed from sodium need.

[0273] 3) Intentional residual sodium gap: The dosage is intentionally calibrated to leave asodium gap — a positive difference between total sodium loss and total sodium delivered by the prescribed sticks. This gap is a designed feature, not an under-dose, and the rationale is explicitly documented in the source: it favors marine-mineral / pH 8.8 absorption pathways over high-dose salt loading, on the premise that the absorption surface is the limiting factor rather than total sodium delivery.

15 **[0274]** 4) Replacement target: 125% of sweat loss over a 4-hour rehydration window (Sawka §G — acknowledged prior art) AND 90% of sweat rate for any future session of similar profile (Sawka §F — acknowledged prior art).

20 **[0275]** A hydration-prescription method in which dosage is bounded by a manufacturer-specified product-form ratio rather than computed solely from physiological need. - A method comprising the deliberate computation and presentation of a “sodium gap” — a residual unmet sodium target — as a designed property of the prescription, accompanied by a stated absorption-pathway rationale. - A method for selecting among multiple form factors of a hydration product (RTD, stick, scoop) by querying a user’s recorded inventory state and applying a deterministic preference ordering.

[0276] Deficit-Banded Autopilot Recheck Cadence

[0277] After computing the user’s hydration deficit percentage, the engine emits an autopilot profile that prescribes when to re-prompt the user for a follow-up reading:

Deficit band	Recheck interval
≥ 4% (Danger)	8 minutes

$\geq 2\%$ (Impaired)	12 minutes
$< 2\%$ (Mild / Optimal)	20 minutes

[0278] The system thereby converts a static post-session calculation into a closed-loop monitoring service. The recheck interval is itself a function of the deficit band, with shorter intervals at higher physiological risk — a control-theoretic adaptive sampling pattern applied to consumer hydration.

5 The recheck profile is part of the invariant output schema (see A.4.1) so it is emitted by all three input modes.

[0279] Inventive claim element: A method wherein a sweat-loss computation produces, in addition to the loss estimate, a recheck-interval prescription whose duration is inversely related to a computed dehydration risk band, enabling a downstream scheduler to convert the calculator into a
10 continuous monitoring loop.

[0280] BSA-Anchored Empirical Estimate Mode

[0281] When the user has no scale, the system computes a population-anchored sweat rate using the multiplicative form: $SR_{final} = SR_{mean[sport]} \times (BSA_{user} / BSA_{nominal}) \times IntensityScale \times HeatFactor \times AcclimFactor$

15 [0282] where: - $SR_{mean[sport]}$ is a sport-specific population mean (Maughan 2007, Δ prior art)
- BSA_{user} is computed from height and weight via Du Bois (Δ prior art) - $BSA_{nominal}$ is a fixed normalization constant (1.9 m^2) representing the typical adult athlete in the cited population studies – $IntensityScale$ is a user-selected 1–5 multiplier mapped to a continuous coefficient - $HeatFactor$ is the climate-strain factor described in A.4.6 – $AcclimFactor$ is the +8% adjustment
20 from A.4.2.

[0283] The novelty is the specific multiplicative composition — five factors chained together to produce a single estimate that approximates the measured-mode output to within a tolerance acceptable for the prescription stage. Each individual factor is in the literature; the engineered combination optimized for converging on the same downstream prescription as the measured path
25 is not.

[0284] A method for estimating per-session sweat rate without a body-mass measurement, comprising the multiplicative composition of (a) a sport-keyed population baseline, (b) a body-surface-area normalization ratio against a nominal adult value, (c) a discrete intensity-level coefficient, (d) a piecewise-linear climate strain factor, and (e) an acclimatization factor, such that

the produced estimate is consumed by an identical downstream prescription stage as the measured-mode output.

[0285] Capped Piecewise-Linear Climate-Strain Factor

[0286] A single-output multiplier derived from temperature and humidity:

5 **[0287]** Temperature contribution: +4% per °C above a 20°C baseline (zero contribution below 20°C)

[0288] Humidity contribution: +0.4% per percentage point of relative humidity above 50% (zero below 50%)

10 **[0289]** Saturation cap: the combined multiplier is hard-capped at 1.6× to prevent unphysiological extrapolation in extreme conditions

[0290] The two contributions are added then capped, producing a bounded, monotonic, continuous adjustment rather than a binary “hot/cold” toggle or a tabular lookup. The cap is set at a level that corresponds approximately to the upper bound of human sweat-rate capacity reported in the literature.

15 **[0291]** A method for computing an environmental-strain multiplier comprising two zero-anchored linear contributions, one per environmental variable, summed and bounded by a saturation cap representing a physiological upper limit.

[0292] Claims

20 **[0293]** A computer-implemented system for producing a hydration prescription tied to a product inventory, comprising: - a measurement-mode computation path receiving at least pre-session and post-session body-mass values; - an estimation-mode computation path receiving body dimensions and an intensity level in lieu of mass measurements; - a unified output module producing, from either path, (a) a sweat-loss volume, (b) a deficit-band classification, (c) a sodium-loss mass, (d) a prescription identifying a count of a manufacturer-specified product unit, and (e) a recheck-interval
25 value; - wherein the prescription count is computed as a function of replacement-water volume and a fixed product-form ratio rather than as a function of the sodium-loss mass alone, producing a defined sodium gap between sodium loss and sodium delivered.

30 **[0294]** A method of generating a hydration recommendation comprising: computing a total sodium loss; selecting a hydration product form factor by querying a user’s recorded inventory under a deterministic preference ordering; computing a recommended product unit count based on a fixed unit-per-water-volume ratio; computing and surfacing to the user a residual sodium-gap

value representing the difference between sodium loss and sodium delivered by the recommendation; and presenting the recommendation with the gap as an intentional property.

[0295] A method of computing predicted electrolyte loss comprising: receiving a single binary acclimatization input; conditional upon said input being asserted, applying within a single
5 computation pass a first multiplicative factor greater than 1.0 to a sweat-volume estimate and a second multiplicative factor less than 1.0 to a sweat-sodium-concentration estimate; combining the adjusted volume and concentration to produce a total sodium-loss prediction.

[0296] A method comprising: computing a hydration-deficit percentage; mapping said percentage to one of at least three risk bands; selecting a recheck interval from a finite set of intervals
10 according to said band, where higher-risk bands map to shorter intervals; emitting said interval as part of a structured output consumable by a downstream scheduler that re-invokes the method after the elapsed interval.

[0297] The mode-invariant output schema of A.4.1.

[0298] Specific values for the deficit thresholds (1%, 2%, 4%), product ratio (12 oz/unit), recheck
15 intervals (8/12/20 min), acclimatization factors (1.08 / 0.7), and climate factor coefficients (4%/°C, 0.4%/RH%, 1.6× cap).

[0299] The form-factor preference ordering RTD → Stick → Scoop.

[0300] The 4-band sodium classification combined with the prescription stage.

[0301] The specific multiplicative composition of A.4.5.

[0302] The sodium-gap mechanism is the engine's most commercially defensible element because
20 it converts a physiological subtlety (absorption-pathway preference) into a defensible software policy that competitors cannot replicate without either (a) admitting their product underdoses sodium, or (b) over-prescribing and creating an absorption ceiling. The dual-direction acclimatization coupling and the deficit-banded recheck cadence both create observable behaviors
25 that are easy to detect in a competitor's app for infringement evidence purposes.

[0303] **Example 2: MARITIME HYDRATION INTELLIGENCE SYSTEM (CRUISE MODE)**

[0304] The software system that produces real-time hydration risk assessments and prescriptive
30 recommendations for cruise-ship guests, using a fusion of (a) a static guest-persona profile, (b) live per-port environmental data resolved against a registry of cruise terminal coordinates, and (c) a maritime-specific scoring model that incorporates variables (sun exposure, alcohol intake, deck

exposure, port-day excursion) absent from terrestrial hydration tools. The system further employs a three-tier resilience strategy ensuring deterministic, plausible output under network or upstream failure.

[0305] Problem solved

- 5 **[0306]** Cruise guests face a hydration profile that no existing terrestrial tool addresses correctly:
- [0307]** Sustained sun exposure at low-latitude ports (often 6–8 hours uninterrupted)
- [0308]** High alcohol intake as a normalized recreational behavior (3+ drinks/day baseline)
- [0309]** Salt-water environment affecting evaporative cooling
- [0310]** Highly intermittent activity (sedentary deck → 4-hour shore excursion → sedentary again)
- 10 **[0311]** Port-day cycle with predictable pre-departure and return windows
- [0312]** No reliable local weather source — guests are at coordinates that may not match any consumer weather app’s nearest-city resolution
- [0313]** Unreliable connectivity — ship Wi-Fi and cellular at port may fail mid-session
- [0314]** Existing fitness/hydration apps assume a stable terrestrial environment with good
- 15 connectivity, regular activity patterns, and no alcohol baseline. They fail outright in maritime use.
- [0315] Cruise-Terminal Coordinate Registry with Deterministic Sun-Hours Derivation**
- [0316]** The system maintains a curated registry of cruise terminal coordinates rather than relying on geocoding to a city centroid. The registry currently includes Miami, Cozumel, Nassau, St. Thomas, Grand Cayman, Juneau, Barcelona — selected for both Caribbean and non-Caribbean
- 20 (Alaska, Mediterranean) coverage.
- [0317]** For each port the API returns not only the raw weather payload but also a derived sun hours metric that does not exist in the upstream weather feed. The derivation is a deterministic function of cloud-cover percentage and weather condition class:

Condition	Cloud cover	Derived sun hours
Condition	Cloud cover	Derived sun hours
Rain / Storm / Snow	any	0.5
Cloudy	≥ 80%	1.5
Partly cloudy	30–80%	4
Clear	< 30%	7

[0318] This sun Hours value becomes a primary input to the downstream hydration risk model. The novelty is the synthesis of a previously-unmeasured “meaningful direct sun exposure” metric from publicly available cloud and condition fields, calibrated specifically for cruise-ship sun exposure (where guests are typically on open decks rather than partially shaded urban environments).

[0319] A method comprising: receiving from a third-party weather service a cloud-cover percentage and a condition class; computing a derived sun-hours value by deterministic mapping of said cloud-cover and condition into a finite set of sun-hour buckets; and providing said derived value to a downstream hydration risk model as if it were a measured environmental parameter.

[0320] Persona-Environment Fusion with Live Override

[0321] The system maintains a set of pre-configured guest personas — currently “Pool-day Guest” and “Excursion Guest” — each carrying a Cruise Session payload with two parts:

[0322] EnvironmentFactors: temp, humidity, deck exposure, sun hours

[0323] GuestSession: pool hours, alcohol drinks, excursion hours, sleep quality

[0324] When the user selects a persona AND a port, the system performs a partial-override merge: live EnvironmentFactors from the port API overwrite the persona’s static environmental fields, while the persona’s GuestSession behavioral fields are preserved. The merge is gated by a matchedLiveEnv predicate ensuring the live data corresponds to the currently-selected port (preventing display of stale data from a previous selection during async fetches).

[0325] After the merge, the evaluate Cruise function recomputes the orb score and recommendations against the merged session.

[0326] A method for producing a personalized recommendation comprising: maintaining a persona record with both behavioral and environmental sub-records; receiving a real-time environmental payload from an external service; merging said payload into the persona record by overwriting only the environmental sub-record while preserving the behavioral sub-record; gating said merge by a predicate verifying that the payload’s location corresponds to the currently-selected location; and recomputing a recommendation against the merged record.

[0327] Maritime-Specific Hydration Erosion Model

[0328] The core scoring algorithm is an “erosion” model: each persona begins with a base hydration score, and a series of categorical “hits” is subtracted to produce a final score that maps to one of four status bands.

[0329] Documented hit values include: Alcohol load: -14 score for ≥ 3 drinks consumed; Heat-index sun load: -18 score for heat index $\geq 103^{\circ}\text{F}$ (heat index from NOAA Rothfus, Δ prior art); Sun-hours load: -8 score for ≥ 4 hours of derived sun exposure; Deck exposure load: -2 score for effect)

5 [0330] The bands and their actions: - Score ≥ 70 : NORMAL - Score 55–69: WATCH - Score 40–54: CAUTION – Score < 40 : RECOVERY_NEEDED $\rightarrow$ triggers a specific scripted recovery cycle: “16 oz water + 1 AForce + 5 min seated cool-down”

[0331] The novelty is the specific selection and weighting of cruise-relevant variables — alcohol, sun-hours, deck exposure — none of which appear together in any consumer hydration model we
10 are aware of, and the mapping of the lowest band to a fixed scripted multi-modal intervention combining hydration, supplementation, and physical posture (seated cool-down).

[0332] A method for assessing hydration status of a person in a maritime recreational environment comprising: maintaining a base score; subtracting from said score a first amount conditional upon an alcohol-consumption count exceeding a threshold; subtracting a second amount conditional
15 upon a derived sun-exposure-hours value exceeding a threshold; subtracting a third amount conditional upon a heat-index value exceeding a threshold; subtracting a fourth amount conditional upon a deck-exposure classification matching a specified outdoor value; mapping the resulting score to one of at least four status bands; and conditional upon the lowest band, presenting a scripted multi-modal recovery cycle comprising fluid volume, electrolyte product unit, and
20 physical-posture instruction.

[0333] Three-Tier Resilience Fallback for Maritime Connectivity

[0334] The system implements a defensive cascade specifically tuned for the unreliable connectivity environment of cruise ships:

[0335] Tier 1 — Client-side identity check: The UI maintains a matchedLiveEnv predicate that
25 suppresses display of any fetched payload whose location does not match the user’s currently-selected port. This prevents the common bug where a slow earlier fetch overwrites a fast later fetch.

[0336] Tier 2 — Server-side TTL cache: A keyed in-memory Map caches per-port environmental

[0337] responses for 10 minutes. The cache key is the port ID. This both respects the upstream API rate limit and provides bounded staleness during repeated guest interactions in the same port window.

[0338] Tier 3 — Deterministic warm-Caribbean fallback: When the upstream weather service is missing an API key, returns a non-200, or exceeds a 4-second timeout, the server returns a hardcoded plausible payload (84°F, 72% humidity, 8 kts wind) representing a typical Caribbean cruise day. This ensures the UI never breaks and the downstream scoring model always has a usable input.

[0339] The fallback is deterministic and labeled in the response, allowing the client to optionally surface the degraded-data state to the user.

[0340] A method for serving environmental data to a recreational maritime client comprising a cascade of: a client-side predicate suppressing payloads not matching the active location selection; a server-side per-location time-to-live cache with a bounded staleness interval; and a deterministic fallback payload representative of the geographic class of the active location, returned and labeled when an upstream service is unavailable, missing credentials, or exceeds a configured timeout.

[0341] Cross-Feature Environmental Bridge to Sweat Engine

[0342] The live environmental payload obtained for Cruise Mode (temperature in °F and relative humidity) is convertible into the ambientTempC and ambientHumidityPct inputs consumed by the Sweat Engine's climate-strain factor (Disclosure A.4.6). This creates an integrated hydration intelligence system in which a guest who returns from a shore excursion can have the same port's environmental conditions automatically pre-fill an excursion sweat-loss calculation, without having to re-enter weather data.

[0343] A combined system in which a per-location environmental payload obtained for the purpose of computing a maritime-recreational hydration risk score is additionally and automatically supplied as input to a separate sweat-loss computation engine, eliminating user re-entry of environmental conditions across the two computations.

[0344] Operator Fleet Dashboard with Weighted Compliance

[0345] A B2B-side aggregation surface for cruise-line operators receives roll-up data from multiple ships in a fleet.

[0346] The summarizeFleet function computes:

[0347] 1) $\text{weightedCompliancePct} = \Sigma(\text{per-ship compliance} \times \text{per-ship total crew}) / \Sigma(\text{per-ship total crew})$. This weights smaller-crew ships proportionally, preventing a single low-crew ship from skewing the fleet average.

[0348] 2) Top-risk ship identification via `highRiskCrewPct` ranking, surfacing the ship requiring intervention first.

[0349] 3) Department breakdown within each ship (Deck Crew, Housekeeping, etc.) for targeted intervention.

[0350] The system thereby converts individual hydration scores into an operator-level operational metric.

[0351] A method for producing a fleet-level hydration-compliance summary across a plurality of vessels comprising: receiving from each vessel a compliance percentage and a crew count; computing a fleet-weighted compliance percentage as the crew-weighted mean; ranking vessels by a high-risk-crew percentage to surface a top-priority intervention target; and producing a per-vessel breakdown by department.

[0352] Independent

[0353] A computer-implemented system for delivering hydration recommendations to a cruise-ship guest, comprising: - a registry of cruise-terminal geographic coordinates; an environmental-data adapter that, for a selected terminal, retrieves an upstream weather payload and derives therefrom a sun-hours value as a deterministic function of cloud-cover and weather-condition fields; a persona store containing at least one record having a behavioral sub-record and an environmental sub-record; - a fusion module that overrides only the environmental sub-record of the selected persona with the retrieved environmental payload, gated by a predicate verifying location identity; - a scoring module that computes a hydration score from the fused record by subtracting categorical hit values keyed on at least an alcohol-consumption count, a sun-hours value, a heat-index value, and a deck-exposure classification; and - a presentation module that maps the score to one of a plurality of bands and emits, for the lowest band, a scripted multi-modal recovery cycle.

[0354] A method for serving environmental data to a maritime client, comprising the cascade enumerated in B.4.4 above.

[0355] A method comprising the deterministic mapping of cloud-cover and condition into a sun-hours metric and providing said metric to a hydration risk model, as enumerated in B.4.1 above.

[0356] A B2B fleet-monitoring system as described in B.4.6 above.

[0357] Dependent

[0358] The specific port registry composition and its mix of Caribbean and non-Caribbean coverage.

5 [0359] The specific sun-hours bucket values.

[0360] The specific score-hit values (-14, -18, -8, -2) and band thresholds (70, 55, 40).

[0361] The 10-minute TTL.

[0362] The 4-second upstream timeout.

[0363] The specific Caribbean fallback payload values (84°F, 72%, 8 kts).

10 [0364] The specific recovery-cycle script ("16 oz water + 1 AForce + 5 min seated cool-down").

[0365] Combination claims integrating B with Disclosure A's Sweat Engine via the bridge of B.4.5.

[0366] Cruise lines represent a B2B contract opportunity that is meaningfully differentiated from any terrestrial fitness/hydration product. The fleet-aggregation surface in particular creates an operator KPI ("weighted hydration compliance") that becomes a contractual deliverable; once a cruise line begins reporting against this metric, switching costs are non-trivial. The maritime-specific scoring weights and the resilience cascade together protect the consumer-side experience from the most common failure modes of competitor apps deployed on cruise ships.

15 [0367] All publications and patent applications mentioned in this specification are herein incorporated by reference to the same extent as if each individual publication or patent application was specifically and individually indicated to be incorporated by reference.

20 [0368] While the invention has been described in connection with various embodiments, it will be understood that the invention is capable of further modifications. This application is intended to cover any variations, uses or adaptations of the invention following, in general, the principles of the invention, and including such departures from the present disclosure as, within the known and
25 customary practice within the art to which the invention pertains.

CLAIMS

What is claimed is:

1. A closed-loop real-time physiological performance operating system, comprising:
continuously monitoring a physiological state, computing a dynamic performance score;
5 delivering compliance-driven commands from a computer-implemented system; and
operating without requiring active user initiation.
2. The system of Claim 1, further comprising real-time physiological and behavioral inputs;
a multi-variable scoring engine; a deterministic AI command enforcement layer; an
autonomous execution engine (“AutoPilot”); and a product-linked intake system
10 (“HydroScan”).
3. The system of Claim 1, further comprising a proactive physiological control system that
continuously evaluates human physiological state and delivers structured behavioral
commands in real time.

ABSTRACT

Provided herein are systems, methods and apparatuses for a closed-loop real-time physiological performance operating system.